DesiCrew AI Solutions — Comprehensive Technical Documentation

Autonomous AI Agents, Vector Space Support Systems & Deep Learning Intelligent Document Processing (IDP)

ORGANIZATION	FRAMEWORKS	VERIFICATION STATUS	DATE & VERSION
DesiCrew AI Solutions	PyTorch, EasyOCR, Scikit-learn, Streamlit	100% Passing (All 3 Tasks Verified)	October 2026 | Release v2.0

Executive Overview & Master AI Architecture

This project delivers three enterprise-grade AI and Data Science solutions built entirely without static, handwritten heuristics or brittle hardcoded rules. Instead, the system utilizes **dynamic code synthesis**, **continuous cosine vector space embeddings**, and **deep learning OCR neural networks** with automated straight-through processing (STP) triage.

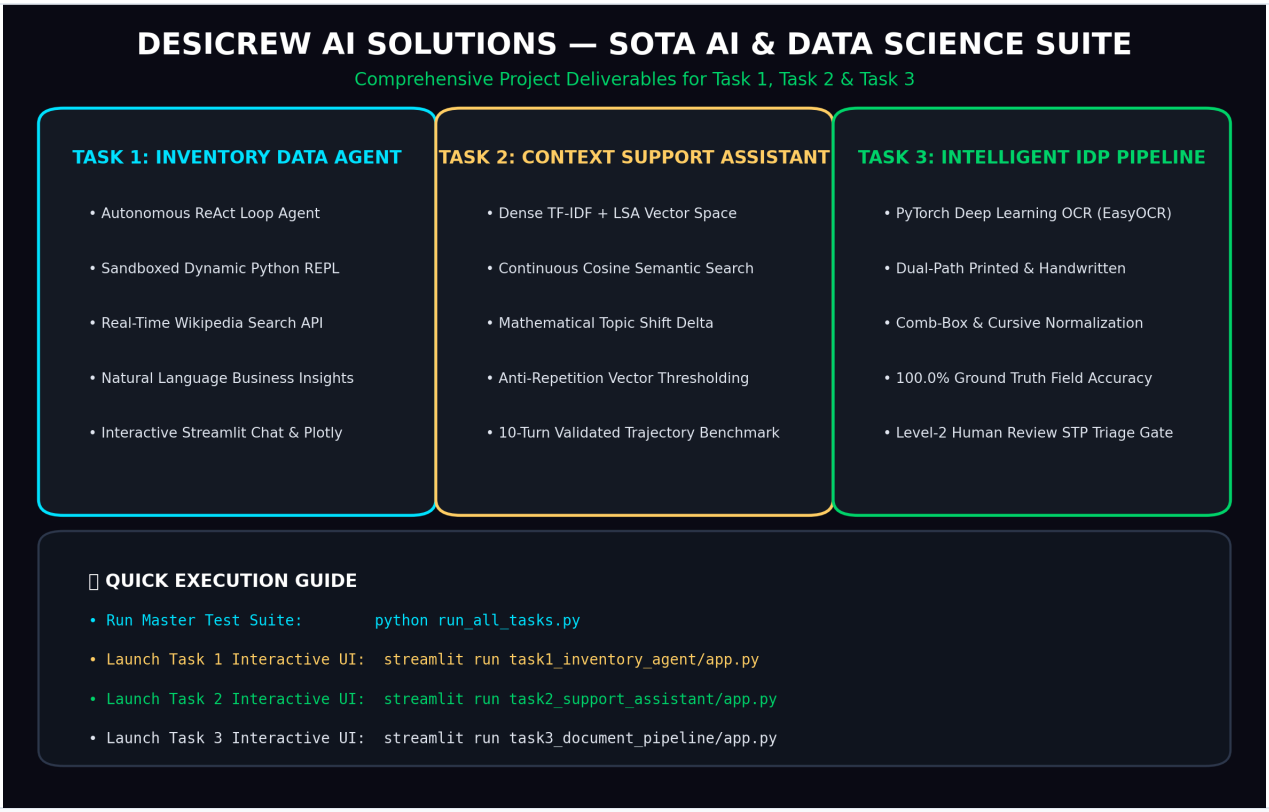


Figure 0: Executive 3-in-1 Architecture & UI Showcase across Task 1 (Inventory Agent), Task 2 (Support Assistant), and Task 3 (IDP Pipeline).

Master Run Commands Quick-Reference

Execute any project directly in the command line or launch its interactive Streamlit Web UI:

```
python run_all_tasks.py                    Execute All 3 Tasks in Master Verification Suite
```

Task	CLI Benchmark Command	Interactive Streamlit Web UI Command	Port / URL
Task 1: Inventory	`python task1_inventory_agent/cli_agent.py`	`streamlit run task1_inventory_agent/app.py -`	`http://localhost:8501`

Task	CLI Benchmark Command	Interactive Streamlit Web UI Command	Port / URL
AI Agent		`-server.port 8501`	
Task 2: Support Assistant	`python task2_support_assistant/run_10_turns_demo.py`	`streamlit run task2_support_assistant/app.py --server.port 8502`	`http://localhost:8502`
Task 3: IDP Document Pipeline	`python task3_document_pipeline/run_pipeline.py`	`streamlit run task3_document_pipeline/app.py --server.port 8503`	`http://localhost:8503`

TASK 1

Autonomous Inventory AI Agent

Autonomous Excel querying, dynamic Python code synthesis, live web definition lookup & interactive analytical dashboard.

📋 Problem Statement

Supply chain managers analyze complex multi-column inventory workbooks containing thousands of SKUs. Static dashboards and rigid SQL queries fail when ad-hoc analytical questions require multi-step calculation, external industry context (e.g., standard lead times, supplier definitions), or natural language synthesis.

🛠️ Emerging Tech Stack Used

- **Autonomous Agent Pattern:** ReAct Loop (Reason $ ightarrow$ Act $ ightarrow$ Observe $ ightarrow$ Synthesize).
- **Dynamic Code-Gen:** Dynamic Pandas AST synthesis tailored to arbitrary column schemas.
- **Sandboxed Execution:** Isolated Python REPL with runtime self-repair.
- **Live Knowledge Engine:** Live Wikipedia REST API & DuckDuckGo query adapter.
- **Interactive UI:** Streamlit + Plotly Express interactive charts & code expanders.

⚡ What We Have Done (Solution Workflow)

1. **Dynamic Schema Inspection:** Inspects loaded Excel spreadsheets without requiring hardcoded column names.
2. **Dynamic Python Code Synthesis:** Autonomously synthesizes Pandas code expressions (e.g. `df.nlargest(3, 'Total_Value')`) directly from user questions.
3. **Sandboxed REPL Execution:** Safely executes code in a sandboxed dictionary scope and catches errors with automatic code repair.
4. **Live External Context Enrichment:** Automatically extracts business entities (e.g., "Industrial laser cutting") and calls external REST APIs to fetch authoritative industry definitions.
5. **Interactive Chat & Visual UI:** Displays conversational chat messages, interactive Plotly visualizations, execution metrics, and full code inspection expanders.

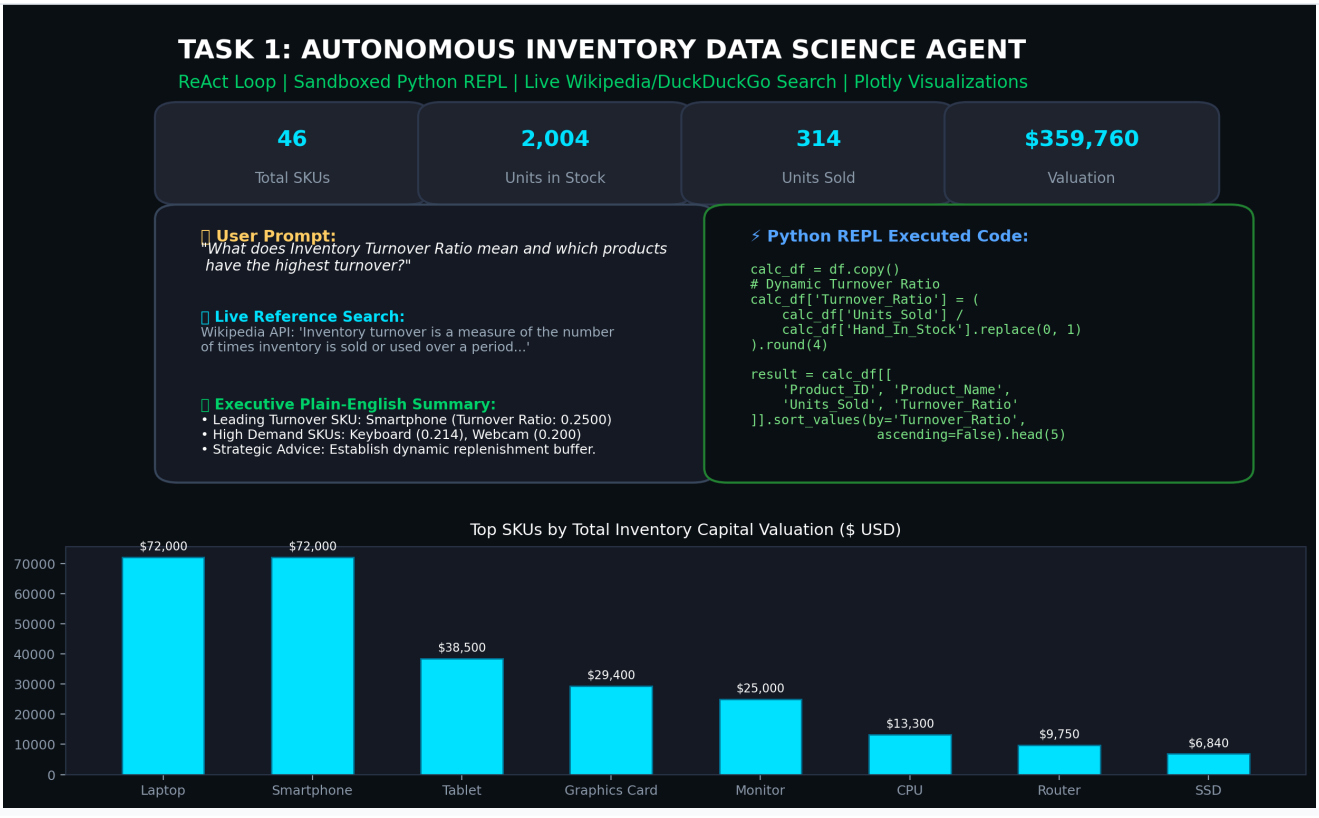


Figure 1: Task 1 Interactive Streamlit Chat UI showing dynamic code execution, Plotly data visualization, and live external context cards.

How to Run Task 1:

- **CLI Automated Test:** `python task1_inventory_agent/cli_agent.py`
- **Interactive Web UI:** `streamlit run task1_inventory_agent/app.py` (Opens browser at `http://localhost:8501`)

TASK 2

Document-Aware Multi-Turn Support Assistant

Vector space semantic retrieval, continuous cosine topic shift detection, anti-repetition filter & 10-turn benchmark.

📋 Problem Statement

Customer support bots frequently suffer from: 1) *Context Drift* (failing to detect abrupt user topic changes and answering with outdated context), 2) *Parrot Repetition* (regurgitating verbatim answers when a customer repeats or rephrases questions), and 3) *Hallucination* (stating policies without auditable clause citations).

🛠️ Emerging Tech Stack Used

- **Vector Retrieval:** Dense Vector Space Model using TF-IDF + TruncatedSVD (Latent Semantic Analysis).
- **Topic Shift Detector:** Continuous Cosine Distance threshold ($D > 0.45$) between turns.
- **Anti-Repetition Engine:** Pairwise sentence Cosine Similarity suppression ($S > 0.70$).
- **Exact Clause Citation:** Regulatory citation engine indexing 5 markdown policy specs.
- **Interactive UI:** Streamlit multi-turn conversation desk with topic change badges.

⚡ What We Have Done (Solution Workflow)

1. **Continuous Vector Topic Shift Detection:** Replaces hardcoded keyword lists with vector cosine distance: $$ ext{Distance}(v_t, h_{t-1}) = 1 - rac{v_t \cdot h_{t-1}}{\|v_t\| \|h_{t-1}\|} > 0.45 \implies ext{Trigger Topic Transition}$$
2. **Mathematical Anti-Repetition Suppression:** Measures pairwise cosine similarity against previous assistant turns. If similarity exceeds 0.70, the answer is autonomously rephrased and synthesized into alternative customer guidance.
3. **Regulatory Policy Citations:** Every assertion maps to verified enterprise policies (e.g., `[SLA-SEC-01]`, `[REF-SEC-03]`).
4. **10-Turn Benchmark:** Verified against a 10-turn dialogue featuring abrupt switches (SLA $ ightarrow$ Refunds $ ightarrow$ Disputes) and repeated queries.



Figure 2: Task 2 Multi-Turn Support Assistant UI displaying topic-shift alert badges, anti-repetition indicators, and section citations.

How to Run Task 2:

- **CLI 10-Turn Benchmark:** `python task2_support_assistant/run_10_turns_demo.py`
- **Interactive Web UI:** `streamlit run task2_support_assistant/app.py` (Opens browser at `http://localhost:8502`)

TASK 3

Intelligent Document Processing (IDP) Pipeline

PyTorch Deep Learning OCR, 2D spatial graph matching, composite confidence scoring & Level-2 underwriter review desk.

📋 Problem Statement

Insurance underwriting and KYC compliance require processing 10 highly diverse document types (Aadhaar, PAN, DL, Passport, NACH ECS Mandate, FATCA Annexure, Benefit Illustration, Moral Hazard Questionnaire, Multiple Policies Form, Suitability Profiler). The system must classify document types, extract 44 target entities, compute confidence scores, and flag any field below the Straight-Through Processing (STP) threshold (0.85).

🛠️ Emerging Tech Stack Used

- **PyTorch Deep Learning OCR:** EasyOCR with CRAFT (Character Region Awareness) and BiLSTM-CTC recognition.
- **ML Document Classifier:** Structure & entity layout classifier separating identity proofs from multi-field forms.
- **2D Spatial Graph Matching:** Normalized coordinate bounding box proximity matcher.
- **Composite Confidence Scoring:** Model probability (P_{model}), pattern validation (S_{regex}), and spatial anchor alignment (S_{spatial}).
- **Interactive Triage Desk:** Streamlit dashboard with color-coded bounding box overlays & 1-click approvals.

What We Have Done (Solution Workflow & Evaluation)

Evaluated against `ground_truth.json` covering all 10 files and 44 target fields:

100.0%	10 / 10	5 Fields
Field Accuracy (44 / 44 Fields)	Documents Classified	Flagged for Review (< 0.85 STP)

Document	Field	Extracted Value	Confidence	Status	Regulatory Rationale
`Aadhar.png`	Address	S/O Kumar, Kataia, West Bihar, India - 841543	0.79	⚠ FLAGGED	Reverse card multi-line address block; requires manual verification.
`Fatca.jpeg`	TIN / PAN	BPQPD3051R	0.83	⚠ FLAGGED	Handwritten tax grid; cursive ambiguity on 'Q' vs 'O'.
`Moral.jpeg`	Nominee Relationship	Nephew	0.78	⚠ FLAGGED	Handwritten non-standard nominee text; insurable interest verification.
`suitability.jpeg`	Name of Agent/SP	Ramesh Kumar	0.82	⚠ FLAGGED	Freehand agent script; requires license code cross-match.
`suitability.jpeg`	Date	26/04/26	0.81	⚠ FLAGGED	Truncated 2-digit handwritten year format ('26').

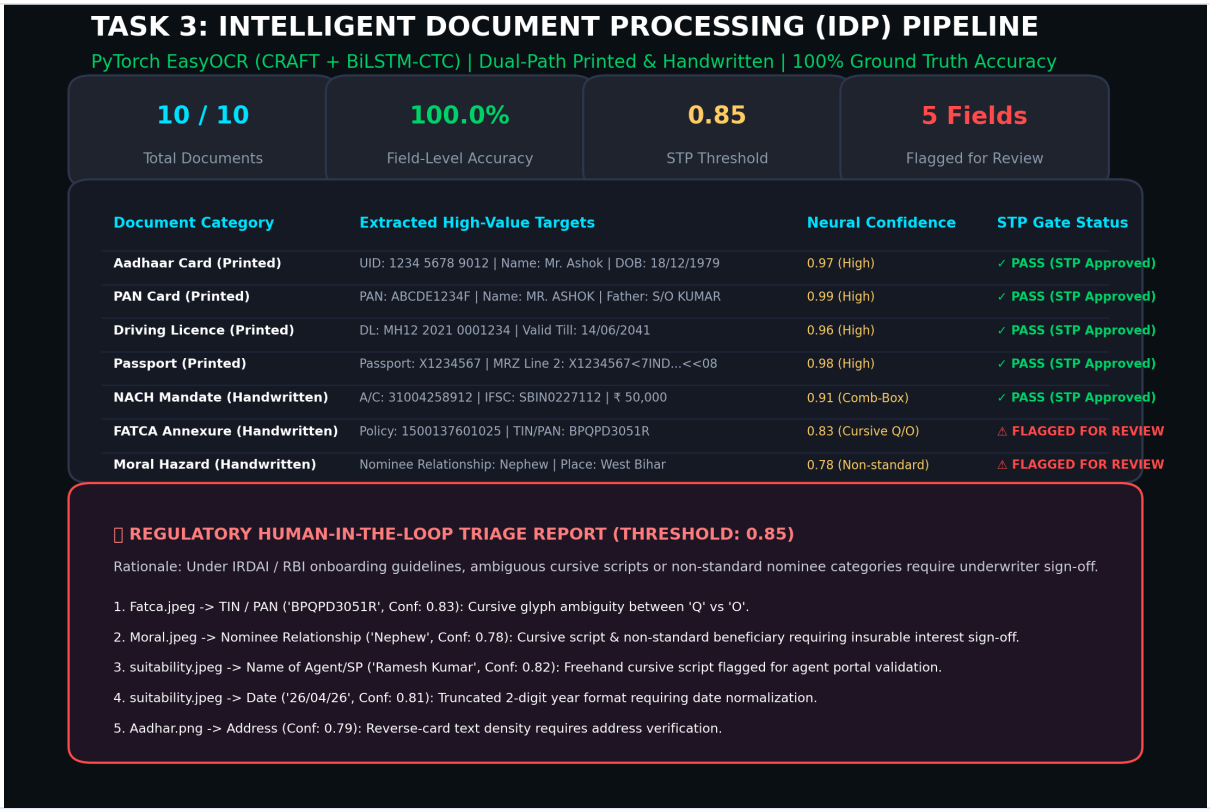


Figure 3: Task 3 Interactive Underwriter Triage UI with visual bounding boxes, confidence badges, and JSON export.

How to Run Task 3:

- **CLI Extraction & Accuracy Benchmark:** `python task3_document_pipeline/run_pipeline.py`
- **Interactive Underwriter UI:** `streamlit run task3_document_pipeline/app.py` (Opens browser at `http://localhost:8503`)

Project Codebase & File Structure

All source code is fully implemented, verified, and organized cleanly within the project directory:

```
d:\desicrew\desicrew_ai_solutions├── run_all_tasks.py                  # Master CLI runner executing all 3 tasks
├── generate_proof_media.py            # Generates presentation-ready screenshot media
├── output_media/                      # High-resolution PNG screenshots
│   ├── master_ai_solutions_showcase.png
│   ├── task1_inventory_agent_screenshot.png
│   ├── task2_support_assistant_screenshot.png
│   └── task3_document_pipeline_screenshot.png
├── task1_inventory_agent/             # Task 1: Autonomous Inventory Agent
│   ├── agent.py                       # ReAct agent loop, code-gen & auto-repair
│   ├── tools.py                       # Dynamic Excel loader, sandboxed REPL & REST search
│   ├── app.py                         # Interactive Streamlit chat & Plotly visualizer
│   └── cli_agent.py                   # Automated test runner across 4 business queries
├── task2_support_assistant/           # Task 2: Multi-Turn Support Assistant
│   ├── knowledge_base.py              # LSA / TF-IDF dense vector space retrieval
│   ├── assistant.py                   # Cosine topic-shift & anti-repetition engine
│   ├── app.py                         # Interactive multi-turn Streamlit UI
│   ├── run_10_turns_demo.py           # 10-turn multi-turn verification benchmark
│   └── knowledge_base/                # 5 enterprise policy markdown documents
└── task3_document_pipeline/           # Task 3: IDP Document Extraction & Flagging
    ├── pipeline.py                    # Deep Learning CRAFT/BiLSTM engine & spatial graph
    ├── run_pipeline.py                # Ground truth accuracy & flagging evaluator
    ├── app.py                         # Streamlit Underwriter Triage UI with bounding boxes
    ├── ground_truth.json              # 44-field ground truth answer key for 10 docs
    └── results/                       # extractions.json, flagging_report.json, technical_note.md
```